

5 Supplementary Strategies

Rebalancing Grid · Stablecoin Grid · Flash Crash Grid

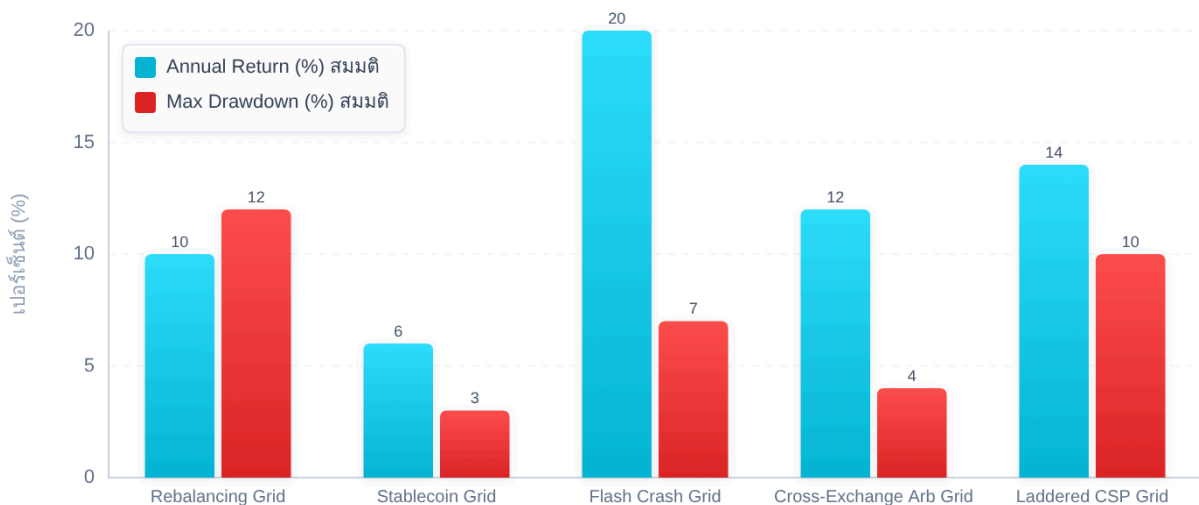
Cross-Exchange Arbitrage Grid · Laddered CSP Grid

แต่ละ strategy ใช้ core grid mechanics แต่ context ต่างกัน

แก่นของ PART นี้

5 strategies นี้ไม่ใช่ "แบบอื่น" ของ grid — ทั้งหมดใช้ core mechanics เดิม (zone, step, Q, Close System) แต่ apply ใน context พิเศษ เพื่อ capture โอกาสที่ standard grid ทำไม่ได้

5 Supplementary Grid Strategies — Return vs Risk Profile (สมมติ)



Flash Crash: return สูงสุดแต่เกิดน้อยครั้ง / Stablecoin: risk ต่ำสุด / Rebalancing: สม่ำเสมอตลอดปี — ตัวเลขเป็น illustration เท่านั้น

เปรียบเทียบ 5 กลยุทธ์เสริม — แต่ละกลยุทธ์เหมาะกับ market regime ที่ต่างกัน

STRATEGY 1

Rebalancing Grid — ทำกำไรจาก Portfolio Drift

ใช้ grid mechanics เพื่อ rebalance portfolio โดยอัตโนมัติ — แทนที่จะ rebalance ทุกเดือนแบบ fixed, grid ทำ continuous rebalancing ตาม price movement:

Rebalancing Grid Setup:

เป้าหมาย: รักษา BTC/USDT = 50%/50% ของ portfolio

Portfolio: \$20,000 total (\$10,000 BTC + \$10,000 USDT)

BTC = \$100k → จำนวน BTC = 0.10

เมื่อ BTC ขึ้น:

BTC value เพิ่ม → % เกิน 50% → grid SELL BTC อัตโนมัติ (TP fills)

ขาย BTC → ได้ USDT → ratio กลับมา 50/50

เมื่อ BTC ลง:

BTC value ลด → % ต่ำกว่า 50% → grid BUY BTC อัตโนมัติ (BUY fills)

ซื้อ BTC → ใช้ USDT → ratio กลับมา 50/50

กำไรจาก: "Buy low, sell high" ทุกครั้งที่ rebalance
→ ได้ทั้ง rebalancing benefit + grid cashflow

```
def rebalancing_grid(total_value, target_btc_pct, btc_price,
step_pct=0.02):
    """
    Grid ที่ทำ portfolio rebalancing
    step_pct: % deviation ก่อน trigger
    """
    target_btc_value = total_value * target_btc_pct
    target_btc_qty = target_btc_value / btc_price

    # Grid levels รอบ target allocation
    levels = []
    for i in range(-5, 6):
        level_price = btc_price * (1 + i * step_pct)
        if i < 0: # BTC ลง → BUY
            levels.append({"type": "BUY", "price": level_price,
                "qty": target_btc_qty * abs(i) *
0.01})
        elif i > 0: # BTC ขึ้น → SELL
            levels.append({"type": "SELL", "price": level_price,
                "qty": target_btc_qty * i * 0.01})
    return levels
```

STRATEGY 2

Stablecoin Grid — ทำกำไรใน Bear Market

เมื่อ BTC ขาลงและ Hurst สูง (trending down) — grid BTC หยุด แต่ยังทำ Stablecoin Grid ได้ (ไม่ต้องถือ BTC):

Stablecoin Grid Options:

1. USDT/USDC Grid: ซื้อ USDT ถูก ขาย USDT แพง (มี depeg event)
Zone: \$0.995-\$1.005 · Step: \$0.0005
กำไรต่อรอบ: $Q \times \$0.0005 - \text{fee}$ (ต่ำมาก แต่ปลอดภัย)
2. Stablecoin Yield + Grid:
USDT ใน Standby Zone (Part 1A) → ผากใน lending protocol

(Aave/Compound)

ได้ yield 4-8% APY บน idle capital

+ เมื่อ BTC กลับมา → นำ USDT ออก deploy grid ใหม่

3. USDT/BTC ที่ระดับ Support แรง:

ถ้าเชื่อว่า BTC มี hard support เช่น \$70k → เปิด grid ที่ \$65k-\$80k
เป็น "Bottom Fishing Grid" ใน Bear Market

```
def stablecoin_grid_yield(standby_capital, lending_apy=0.05,
grid_days=90):
    """
    คำนวณ yield จาก idle capital ใน bear market
    """
    daily_rate = (1 + lending_apy) ** (1/365) - 1
    total_yield = standby_capital * ((1 + daily_rate) **
grid_days - 1)
    return total_yield

# ตัวอย่าง: Standby $6,000 · APY 5% · 90 วัน
# Yield = $6,000 × ((1.000133)^90 - 1) = $6,000 × 1.2% = $73.5
```

STRATEGY 3

Flash Crash Grid — Capture Extreme Dips

Flash Crash Grid วางล่วงหน้าในช่วง "ต่ำผิดปกติ" ที่ตลาดน่าจะ recover ได้ภายใน 24-72 ชั่วโมง:

Flash Crash Grid:

ลักษณะ Flash Crash ที่เหมาะ:

- BTC ลง > 15% ภายใน 1 ชั่วโมง (liquidation cascade)
[เหตุผล 15%: ใต้ระดับนี้ market makers มักเข้าซื้อ + liquidation ล้างแล้ว
+ ราคาต่ำกว่า fair value ชัดเจน]
- Volume spike > 5× average
- ระดับ macro เหมือนเดิม (on-chain ยังแข็งแกร่ง)
- ไม่มี fundamental bad news

Setup:

วาง BUY orders ทันทีหลัง crash (ไม่ใช่ก่อน crash)

Zone: ราคาปัจจุบัน (หลัง crash) ±5%

Step: ATR ปัจจุบัน (อาจสูงมาก)

Close System: ใช้ capital ที่เตรียมไว้ในส่วน "Flash Crash Reserve" เท่านั้น

```
class FlashCrashGrid:
    def __init__(self, crash_capital_pct=0.10,
recovery_hours=72):
        """
        crash_capital_pct: % ของ total capital สำหรับ flash crash
        recovery_hours: เวลาที่คาดว่าจะ recover ได้
        """
        self.capital_pct = crash_capital_pct
        self.recovery_hours = recovery_hours

    def is_flash_crash(self, drop_pct, volume_ratio,
time_elapsed_hours):
        """
        ตรวจสอบว่าเป็น flash crash หรือ trend reversal
        """
        is_large_drop    = drop_pct > 0.15          # ลง > 15%
        is_fast           = time_elapsed_hours < 2   # เร็ว < 2 ชั่วโมง
        is_vol_spike      = volume_ratio > 3.0       # volume 3x
ปกติ
        return is_large_drop and is_fast and is_vol_spike
```

Risk: Flash Crash กลายเป็น Trend Reversal → Close System รักษา inventory

TP: ปิด grid เมื่อ recover 50% ของ crash depth หรือภายใน recovery_hours

ถ้าไม่ recover ภายใน recovery_hours: ลด inventory 50% เพื่อ free capital → ถือ BTC ที่เหลือในฐานะ long-term หรือโอนเข้า Stablecoin Grid (Strategy 2)

STRATEGY 4

Cross-Exchange Arbitrage Grid

เมื่อ BTC price บน 2 exchanges ต่างกัน — ใช้ grid ทั้งสองฝั่งพร้อมกัน capture spread:

Cross-Exchange Grid:

Exchange A (เช่น Binance): BTC = \$100,050

Exchange B (เช่น Bybit): BTC = \$99,950

Spread = \$100 (ต่าง 0.1%)

Grid Strategy:

Exchange A: SELL BTC @ \$100,050 (ขายแพง)

Exchange B: BUY BTC @ \$99,950 (ซื้อถูก)

Net profit per cycle: \$100 - fees

```
def cross_exchange_spread_grid(price_a, price_b, q, fee_a,
fee_b):
    """
    คำนวณกำไรจาก cross-exchange spread
    """
    spread = abs(price_a - price_b)
    fee_cost = q * (price_a * fee_a + price_b * fee_b)
    net_profit = q * spread - fee_cost

    return {
        "gross_spread": spread,
        "fee_cost": fee_cost,
        "net_profit": net_profit,
        "profitable": net_profit > 0
    }
```

ความท้าทาย:

1. Transfer BTC ระหว่าง exchange ใช้เวลา (10-30 นาที) → spread อาจหายไปแล้ว
2. ต้องมี capital บน ทั้งสอง exchange ตลอด
3. ค่า withdrawal fee + network fee กินกำไร
4. Slippage เมื่อ Q ใหญ่

วิธีแก้: ใช้ Perpetual บน Exchange A (short) + Spot บน Exchange B (long)

→ ไม่ต้อง transfer, ใช้ funding rate เป็น P&L indicator

STRATEGY 5

Laddered CSP Grid — ขาย Put เป็น Grid

แทนที่จะวาง BUY limit order ทั้งหมดพร้อมกัน — ขาย Put options เป็น "ladder" เพื่อรับ premium + ซื้อ BTC ในราคาที่ต้องการ (จาก Part 7):

Laddered CSP Grid:

ตัวอย่าง: BTC = \$100k · Grid capital \$30,000 · 5 levels

Level 1: SELL PUT \$98k, expiry 2W, premium \$600

Level 2: SELL PUT \$95k, expiry 2W, premium \$450

Level 3: SELL PUT \$92k, expiry 4W, premium \$700

Level 4: SELL PUT \$88k, expiry 4W, premium \$550

Level 5: SELL PUT \$84k, expiry 8W, premium \$800

Total premium = \$3,100 (รับทันที)

ถ้าไม่มี level ถูก assign ตลอด 8 สัปดาห์:

Income = \$3,100 premium = 10.3% บน \$30,000 capital ใน 8 สัปดาห์

ถ้า BTC ลงมา \$95k (ถึง Level 2):

Level 1: expire worthless (+\$600 เก็บ)

Level 2: assign → ซื้อ BTC \$95k (กำหนดไว้ล่วงหน้า)

Level 3-5: ยังคง pending

ข้อดีเทียบ Limit Order Grid:

- + รับ premium แม้ไม่มี fill
- + Effective buy price ต่ำกว่า (strike - premium)
- + Capital ไม่ lock ทั้งหมด (แค่ reserved สำหรับ worst-case)

ข้อจำกัด:

- ต้องการ Options liquidity (Deribit/OKX)
- Expiry management ซับซ้อน
- ถ้า BTC crash ลงเร็ว → ทุก level assign พร้อมกัน

```
def laddered_csp_grid(grid_levels, btc_price, put_premiums):
```

```
    """
```

```
    คำนวณ total income และ effective buy prices
```

```
    """
```

```
    results = []
```

```
    for price, premium in zip(grid_levels, put_premiums):
```

```
        effective_cost = price - premium
```

```
        results.append({
```

```
            "strike": price,
```

```
            "premium": premium,
```

```
        "effective_cost": effective_cost,  
        "discount_pct": premium / price * 100  
    })  
    return results
```

แบบฝึกหัด Part 8 — Quick Review

- ▶ ข้อ 1: ใน Bear Market ที่ BTC downtrend ชัด — Strategy ไหนในทั้ง 5 ข้อที่เหมาะสมที่สุด?
- ▶ ข้อ 2: Cross-Exchange Grid กำไรจากอะไร และทำไมถึงทำได้ยากในทางปฏิบัติ?
- ▶ ข้อ 3: Laddered CSP Grid ให้ effective buy price ต่ำกว่า Limit Order Grid อย่างไร?
- ▶ ข้อ 4: Flash Crash Grid ต่างจาก Standard Buy-Only Grid อย่างไร และทำไมต้องใช้ capital แยก (Flash Crash Reserve)?